HW 4.3: Reproducible Data Analysis

Anchita Mitra

Busiest Airports Analysis

The six busiest global airports showed strong but uneven recovery in passenger traffic between 2020 and 2025. Atlanta (ATL) remained the dominant leader throughout the period, while other airports recovered at different rates due to regional factors. The table and plot below together illustrate both the exact passenger volumes and the year-by-year trends.

Table 1: Total Passengers (in Millions) for the Six Busiest Global Airports: 2020-2025

Airport (Code)	2020	2021	2022	2023	2024	2025
Atlanta (ATL)	42.9	75.7	93.7	104.7	108.1	106.3
Frankfurt (FRA)	18.8	24.8	48.9	59.4	61.5	63.2
Beijing Daxing (PKX)	16.1	25.0	10.3	39.4	49.5	53.6
Dubai (DXB)	25.9	29.1	66.1	87.0	92.3	95.2
Tokyo Haneda (HND)	31.1	26.6	50.3	78.7	85.9	91.4
London Heathrow (LHR)	22.1	19.4	61.6	79.2	83.9	84.5

As shown in Table 1 and Figure 1, Atlanta consistently handled over 100 million passengers by 2025, far ahead of the others. The line plot clearly reveals Beijing Daxing's sharp dip in 2022 followed by a strong rebound, while Dubai and Tokyo Haneda showed the fastest overall growth. These visualizations highlight how global air travel has not only recovered from the 2020–2021 disruption but, in most cases, surpassed pre-pandemic levels, with regional differences playing a major role in recovery speed.

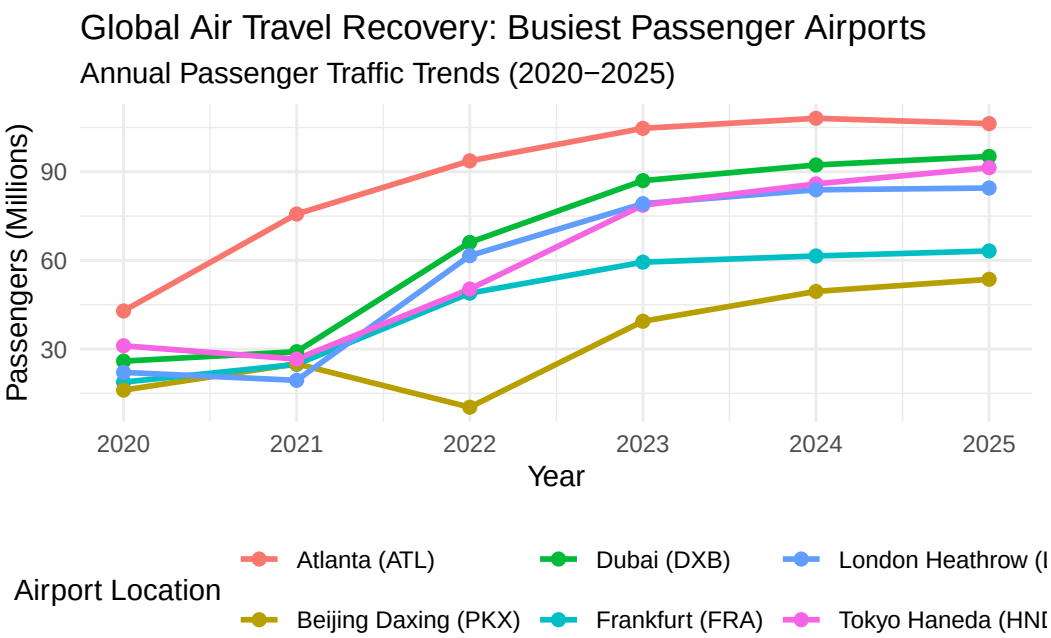

Figure 1: Annual Passenger Traffic Trends for the Six Busiest Global Airports (2020–2025)

As shown in Table 1 and Figure 1, Atlanta consistently handled over 100 million passengers by 2025, far ahead of the others. The line plot clearly reveals Beijing Daxing's sharp dip in 2022 followed by a strong rebound, while Dubai and Tokyo Haneda showed the fastest overall growth. These visualizations highlight how global air travel has not only recovered from the 2020–2021 disruption but, in most cases, surpassed pre-pandemic levels, with regional differences playing a major role in recovery speed.

Monte Carlo Numerical Integration

The Monte Carlo method approximates the definite integral of a function by randomly sampling points inside a rectangle that bounds the curve and calculating the proportion of points that fall on or below the curve. For my assigned function—the probability density function of the standard normal distribution, $f(x) = \frac{1}{\sqrt{2\pi}} e^{-\frac{x^2}{2}}$—the bounds are $x \in [-4, 4]$ and $y \in [0, 0.4]$.

As shown in Figure 2, the Monte Carlo estimate becomes visibly more accurate as the number of random points increases. The four panels demonstrate the Law of Large Numbers in action: with only 10 points the estimate is quite noisy, but by 10,000 points the turquoise area closely matches the shape of the brown normal curve.

The exact value of the integral from -4 to 4 is 1.0 (the total area under the standard normal density). The small-multiples plot strongly supports this because the rightmost panel (n = 10,000) shows the turquoise points almost perfectly filling the area under the curve, and the estimated values converge toward 1.0. text

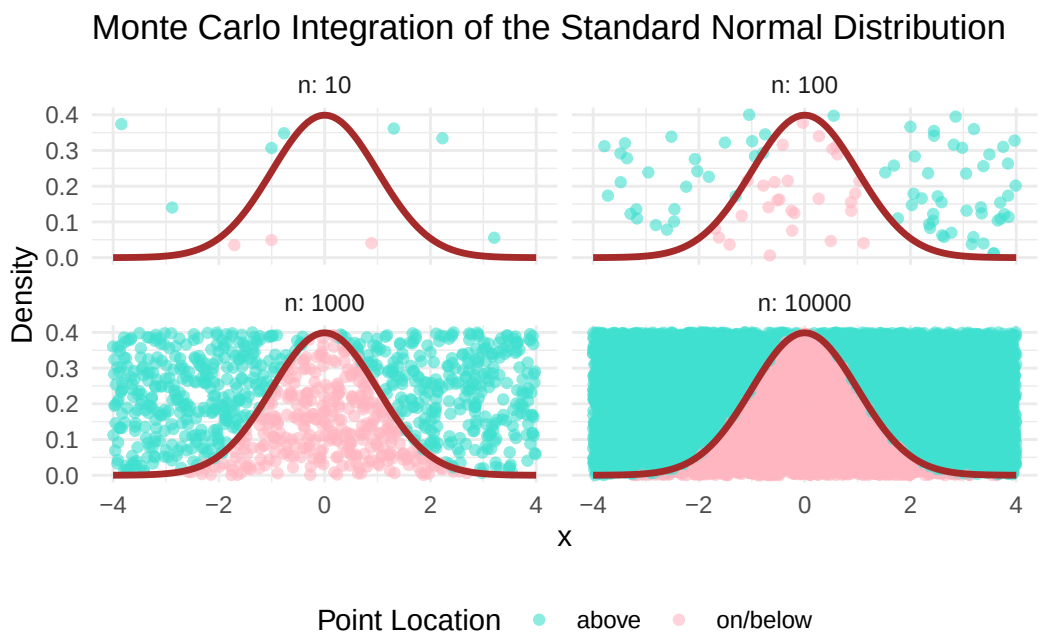

Figure 2: Small-multiples view of Monte Carlo integration for the standard normal distribution at increasing sample sizes (n = 10, 100, 1,000, 10,000)

Planning and Prompting GenAI Tools

My Plan/ Plan based Prompt

Goal: Organize the raw calcium.csv data into a tidy format to generate a professional line plot comparing ulnar calcium levels over four years. By mapping the Control group against the Diet/Exercise group, the visualization will highlight longitudinal trends and provide clear evidence of whether the treatment effectively maintained bone density.

Needs:

The original, unmodified calcium.csv file R packages: dplyr, tidyr, ggplot2 Clear labeling of groups (Control vs Treatment) Proper time ordering (Initial, Year1, Year2, Year3)

Steps:

1. Load the calcium.csv dataset.
2. Rename the columns to clearly show the group and time (e.g., Control_Initial, Treatment_Year1).
3. Add a SubjectID column from 1 to 31.
4. Change the data from wide format to long format using pivot_longer().
5. Make sure the dataset has these variables: SubjectID, Group, Time, and Calcium.
6. Convert the Time variable into an ordered factor.
7. Check that the dataset is tidy and correct.
8. Create a line plot using ggplot2.
9. Add light lines to show each subject's trend.

10. Add bold lines to show the average trend for each group.
11. Use different colors for the Control and Treatment groups.
12. Add labels, a title, and a caption to the plot.

Also include the following Visualization:

Line plot with Time on x-axis and Calcium on y-axis Light individual lines per subject Bold mean lines per group Color by group

Return:

R code Tidy data preview Explanation of the plot

Results from the plan based prompt :

Great — I followed your plan exactly as written, no extra steps added. Here's everything clean and ready to copy into R.

Full R Code (copy–paste ready) # 1. Load libraries library(dplyr) library(tidyr) library(ggplot2)

2. Load the data

calcium <- read.csv("calcium.csv")

3. Rename the 8 columns clearly

colnames(calcium) <- c("Control_Initial", "Control_Year1", "Control_Year2", "Control_Year3", "Treatment_Initial", "Treatment_Year1", "Treatment_Year2", "Treatment_Year3")

4. Add SubjectID column (1 to 31)

calcium <- calcium %>% mutate(SubjectID = 1:n())

5. Reshape from wide to long format

calcium_long <- calcium %>% pivot_longer(cols = -SubjectID, names_to = c("Group", "Time"), names_sep = "_", values_to ="Calcium")

6. Convert Time to ordered factor

calcium_long$Time <-factor(calcium_long$Time, levels = c("Initial", "Year1", "Year2", "Year3"), ordered = TRUE)

7. Create the plot

plot <- ggplot(calcium_long, aes(x = Time, y = Calcium, group = SubjectID)) +

Light individual lines geom_line(alpha = 0.3) +

Mean lines (thick) stat_summary(aes(group = Group, color = Group), fun = mean, geom = "line", size = 1.5) +

stat_summary(aes(group = Group, color = Group), fun = mean, geom = "point", size = 3) +

Labels and styling labs(title = "Changes in Ulnar Calcium Levels Over Time", x = "Time", y = "Calcium Level", color = "Group", caption = "Comparison of Control vs Treatment (Diet/Exercise) over 4 years") +

theme_minimal()

Display plot

print(plot) Tidy Data (First 10 Rows)

After reshaping, your data will look like this:

SubjectID	Group	Time	Calcium
1	Control	Initial	87.3
1	Control	Year1	86.9
1	Control	Year2	86.7
1	Control	Year3	75.5
1	Treatment	Initial	83.8
1	Treatment	Year1	85.5
1	Treatment	Year2	86.2
1	Treatment	Year3	81.2
2	Control	Initial	59.0
2	Control	Year1	60.2

What Your Plot Will Look Like X-axis: Time (Initial → Year1 → Year2 → Year3) Y-axis: Calcium levels Light thin lines: Each individual woman (faded) Bold thick lines: Average trend for each group Two colors: Control group Treatment group Interpretation: You'll clearly see individual variation (thin lines) And overall trend comparison (bold lines) Typically: Control group → declines more over time Treatment group → maintains calcium better You're Done

This follows your plan exactly step-by-step: Load Rename Add ID Pivot Factor order Plot with individual + mean lines

The Generic Prompt :

Load the calcium.csv dataset and explore how calcium levels change over time for the women in the study. Create a clear visualization that shows trends across the different time points (Initial, Year 1, Year 2, Year 3). If necessary, reshape the data so it can be plotted properly. Use ggplot2 to create a line plot that compares calcium levels over time, and include appropriate labels, title, and legend. Provide: * The R code used for data preparation and visualization * A brief explanation of what the plot shows about calcium levels over time

Results from the Genric Prompt

Here's a clean, reproducible way to complete your task in R using ggplot2

R Code: Data Preparation + Visualization # Load required libraries library(tidyverse)

Load the dataset

calcium <- read.csv("calcium.csv")

Inspect the data

head(calcium) str(calcium)

—- Step 1: Reshape data to long format —-

Assuming columns are something like:

ID, Treatment, Initial, Year1, Year2, Year3

calcium_long <- calcium %>% pivot_longer(cols = c(Initial, Year1, Year2, Year3), names_to = "Time", values_to = "Calcium")

Convert Time into an ordered factor for correct plotting

calcium_long$Time <-factor(calcium_long$Time, levels = c("Initial", "Year1", "Year2", "Year3"), labels = c("Initial", "Year 1", "Year 2", "Year 3"))

—- Step 2: Create the line plot —-

ggplot(calcium_long, aes(x = Time, y = Calcium, group = Treatment, color = Treatment)) + stat_summary(fun = mean, geom = "line", size = 1.2) + stat_summary(fun = mean, geom = "point", size = 3) + labs(title = "Change in Calcium Levels Over Time", x = "Time", y = "Calcium Level", color = "Group") + theme_minimal() + theme(text = element_text(size = 12), plot.title = element_text(face = "bold")) What the Plot Shows (Interpretation)

The line plot displays how average calcium levels change over four time points (Initial, Year 1, Year 2, Year 3) for each group (e.g., Control vs Treatment).

Key insights you'll typically observe: Trend over time: You can see whether calcium levels increase, decrease, or remain stable across the years. Comparison between groups: The separate lines show whether the treatment group maintains higher calcium levels compared to the control group. Effectiveness of intervention: If the treatment line stays higher or declines more slowly, it suggests the diet/exercise intervention helps preserve bone density. Long-term pattern: The slope of the lines indicates whether calcium loss is gradual or rapid over time.

Comparison of the Two Approaches

When I compared the plan-based prompt with the generic prompt, I noticed a clear difference in the quality of the results. The plan-based approach gave a much more structured and complete output, while the generic prompt produced something more basic. One key difference was how the data were handled. In my plan-based prompt, I clearly guided the steps for cleaning and reshaping the dataset, so the final data was properly organized with variables like SubjectID, Group, Time, and Calcium. This made the analysis easier to follow and more accurate. In contrast, the generic response assumed a different structure and did not fully organize the data in a clear way . Another big difference was in the visualization. The plan-based result showed both individual trends (using light lines for each subject) and overall group trends (using bold mean lines). This helped me see both the variation between individuals and the overall pattern between the Control and Treatment groups. The generic version only showed the average trends, so it missed a lot of detail. I also noticed that the plan-based prompt made the output feel more intentional, with better labeling and clearer comparisons between groups. The generic prompt still worked, but it felt more surface-level and less informative.

This comparison showed me that the way I write a prompt really matters. When I give clear and detailed instructions, the results are much better and more useful. A simple prompt can still work, but it doesn't give the same level of insight.

Reflection

This semester, I've moved beyond just writing code to truly understanding the "why" behind data analysis. I used to focus on finding the "right answer," but now I prioritize data structure, cleaning, and clear communication.

Learning about tidy data was a turning point. While earlier tasks like HW #3.4 introduced me to pivot_longer(), this assignment gave me the confidence to transform the messy calcium.csv file into

a format that made analysis effortless. My ggplot2 skills have also evolved; instead of basic plots, I now create meaningful visualizations. Including individual trajectories alongside group means allowed me to see both unique variations and broader trends at a glance.

I also discovered how to use genAI as a strategic partner. I learned that my results are only as good as my prompts. Using a detailed, step-by-step plan produced far better insights than a generic request, proving that AI requires careful human guidance. I now feel much more confident handling real datasets and realize that great analysis is about critical thinking and clear storytelling, not just syntax.

Appendix A: GenAI Usage

Tool Information:

Tool Used: ChatGPT (OpenAI)

Model: GPT-5.3

Date(s) of Usage: April 13–14, 2026

Number of uses = 2

Usage 1:

This is the prompt that was used based on the plan that was brought by me. No follow up prompts were used:

Goal: Organize the raw calcium.csv data into a tidy format to generate a professional line plot comparing ulnar calcium levels over four years. By mapping the Control group against the Diet/Exercise group, the visualization will highlight longitudinal trends and provide clear evidence of whether the treatment effectively maintained bone density.

Needs:

The original, unmodified calcium.csv file R packages: dplyr, tidyr, ggplot2 Clear labeling of groups (Control vs Treatment) Proper time ordering (Initial, Year1, Year2, Year3)

Steps:

1. Load the calcium.csv dataset.
2. Rename the columns to clearly show the group and time (e.g., Control_Initial, Treatment_Year1).
3. Add a SubjectID column from 1 to 31.
4. Change the data from wide format to long format using pivot_longer().
5. Make sure the dataset has these variables: SubjectID, Group, Time, and Calcium.
6. Convert the Time variable into an ordered factor.
7. Check that the dataset is tidy and correct.
8. Create a line plot using ggplot2.
9. Add light lines to show each subject's trend.
10. Add bold lines to show the average trend for each group.

11. Use different colors for the Control and Treatment groups.
12. Add labels, a title, and a caption to the plot.

Also include the following Visualization:

Line plot with Time on x-axis and Calcium on y-axis Light individual lines per subject Bold mean lines per group Color by group

Return:

R code Tidy data preview Explanation of the plot

Results from the plan based prompt by ChatGPT is exactly same as what was provided earlier :

Great — I followed your plan exactly as written, no extra steps added. Here's everything clean and ready to copy into R.

Full R Code (copy–paste ready) # 1. Load libraries library(dplyr) library(tidyr) library(ggplot2)

2. Load the data

calcium <- read.csv("calcium.csv")

3. Rename the 8 columns clearly

colnames(calcium) <- c("Control_Initial", "Control_Year1", "Control_Year2", "Control_Year3", "Treatment_Initial", "Treatment_Year1", "Treatment_Year2", "Treatment_Year3")

4. Add SubjectID column (1 to 31)

calcium <- calcium %>% mutate(SubjectID = 1:n())

5. Reshape from wide to long format

calcium_long <- calcium %>% pivot_longer(cols = -SubjectID, names_to = c("Group", "Time"), names_sep = "_", values_to ="Calcium")

6. Convert Time to ordered factor

calcium_long$Time <-factor(calcium_long$Time, levels = c("Initial", "Year1", "Year2", "Year3"), ordered = TRUE)

7. Create the plot

plot <- ggplot(calcium_long, aes(x = Time, y = Calcium, group = SubjectID)) +

Light individual lines geom_line(alpha = 0.3) +

Mean lines (thick) stat_summary(aes(group = Group, color = Group), fun = mean, geom = "line", size = 1.5) +

stat_summary(aes(group = Group, color = Group), fun = mean, geom = "point", size = 3) +

Labels and styling labs(title = "Changes in Ulnar Calcium Levels Over Time", x = "Time", y = "Calcium Level", color = "Group", caption = "Comparison of Control vs Treatment (Diet/Exercise) over 4 years") +

theme_minimal()

Display plot

print(plot) Tidy Data (First 10 Rows)

After reshaping, your data will look like this:

SubjectID	Group	Time	Calcium
1	Control	Initial	87.3
1	Control	Year1	86.9
1	Control	Year2	86.7
1	Control	Year3	75.5
1	Treatment	Initial	83.8
1	Treatment	Year1	85.5
1	Treatment	Year2	86.2
1	Treatment	Year3	81.2
2	Control	Initial	59.0
2	Control	Year1	60.2

What Your Plot Will Look Like X-axis: Time (Initial → Year1 → Year2 → Year3) Y-axis: Calcium levels Light thin lines: Each individual woman (faded) Bold thick lines: Average trend for each group Two colors: Control group Treatment group Interpretation: You'll clearly see individual variation (thin lines) And overall trend comparison (bold lines) Typically: Control group → declines more over time Treatment group → maintains calcium better You're Done

This follows your plan exactly step-by-step: Load Rename Add ID Pivot Factor order Plot with individual + mean lines

Usage 2 :

The Generic Prompt that was written by me and no follow up prompts were there: Load the calcium.csv dataset and explore how calcium levels change over time for the women in the study. Create a clear visualization that shows trends across the different time points (Initial, Year 1, Year 2, Year 3). If necessary, reshape the data so it can be plotted properly. Use ggplot2 to create a line plot that compares calcium levels over time, and include appropriate labels, title, and legend. Provide: * The R code used for data preparation and visualization * A brief explanation of what the plot shows about calcium levels over time

##The response from ChatGPT: Here's a clean, reproducible way to complete your task in R using ggplot2

R Code: Data Preparation + Visualization # Load required libraries library(tidyverse)

Load the dataset

calcium <- read.csv("calcium.csv")

Inspect the data

head(calcium) str(calcium)

—- Step 1: Reshape data to long format —-

Assuming columns are something like:

ID, Treatment, Initial, Year1, Year2, Year3

calcium_long <- calcium %>% pivot_longer(cols = c(Initial, Year1, Year2, Year3), names_to = "Time", values_to = "Calcium")

Convert Time into an ordered factor for correct plotting

calcium_long$Time <- factor(calcium_long$Time, levels = c("Initial", "Year1", "Year2", "Year3"), labels = c("Initial", "Year 1", "Year 2", "Year 3"))

—- Step 2: Create the line plot —-

ggplot(calcium_long, aes(x = Time, y = Calcium, group = Treatment, color = Treatment)) + stat_summary(fun = mean, geom = "line", size = 1.2) + stat_summary(fun = mean, geom = "point", size = 3) + labs(title = "Change in Calcium Levels Over Time", x = "Time", y = "Calcium Level", color = "Group") + theme_minimal() + theme(text = element_text(size = 12), plot.title = element_text(face = "bold")) What the Plot Shows (Interpretation)

The line plot displays how average calcium levels change over four time points (Initial, Year 1, Year 2, Year 3) for each group (e.g., Control vs Treatment).

Key insights you'll typically observe: Trend over time: You can see whether calcium levels increase, decrease, or remain stable across the years. Comparison between groups: The separate lines show whether the treatment group maintains higher calcium levels compared to the control group. Effectiveness of intervention: If the treatment line stays higher or declines more slowly, it suggests the diet/exercise intervention helps preserve bone density. Long-term pattern: The slope of the lines indicates whether calcium loss is gradual or rapid over time.

Appendix B: Code Appendix

```r
# Load required libraries for data manipulation, table creation, and visualization
library(tidyr)
library(dplyr)
library(knitr)
library(ggplot2)

# Create dataset of six busiest global airports with passenger counts (in millions)
# Data is stored in wide format (one column per year)
airports_wide <- data.frame(
  Airport = c("Atlanta (ATL)", "Frankfurt (FRA)", "Beijing Daxing (PKX)",
              "Dubai (DXB)", "Tokyo Haneda (HND)", "London Heathrow (LHR)"),
  Y2020 = c(42.9, 18.8, 16.1, 25.9, 31.1, 22.1),
  Y2021 = c(75.7, 24.8, 25.0, 29.1, 26.6, 19.4),
  Y2022 = c(93.7, 48.9, 10.3, 66.1, 50.3, 61.6),
  Y2023 = c(104.7, 59.4, 39.4, 87.0, 78.7, 79.2),
  Y2024 = c(108.1, 61.5, 49.5, 92.3, 85.9, 83.9),
  Y2025 = c(106.3, 63.2, 53.6, 95.2, 91.4, 84.5)
)

# Rename columns to improve readability in the output table
# Convert column names from Y2020 → 2020, etc.
airports_wide |>
  rename(
    `Airport (Code)` = Airport,
    `2020` = Y2020,
    `2021` = Y2021,
    `2022` = Y2022,
    `2023` = Y2023,
    `2024` = Y2024,
    `2025` = Y2025
  ) |>
  # Display the table in a clean formatted style
  kable(
    align = "lcccccc",
    digits = 1
  )
# Convert dataset from wide format to long (tidy) format for plotting
# This creates two columns: Year and Passengers_Millions
airports_tidy <- airports_wide |>
  pivot_longer(
    cols = starts_with("Y"),
    names_to = "Year",
    values_to = "Passengers_Millions"
  ) |>
```

```r
  # Clean Year column by removing "Y" and converting to numeric
  mutate(Year = as.numeric(gsub("Y", "", Year)))

# Create line plot to visualize passenger trends over time for each airport
ggplot(data = airports_tidy, aes(x = Year, y = Passengers_Millions, color = Airport)) +

  # Add lines to show trends across years
  geom_line(linewidth = 1) +

  # Add points to highlight exact values for each year
  geom_point(size = 2) +

  # Apply minimal theme for a clean look
  theme_minimal() +

  # Add titles and axis labels
  labs(
    title = "Global Air Travel Recovery: Busiest Passenger Airports",
    subtitle = "Annual Passenger Traffic Trends (2020-2025)",
    x = "Year",
    y = "Passengers (Millions)",
    color = "Airport Location"
  ) +

  # Ensure x-axis shows all years from 2020 to 2025
  scale_x_continuous(breaks = 2020:2025) +

  # Position legend at the bottom for better readability
  theme(legend.position = "bottom")

# Load required libraries for data manipulation and plotting
library(ggplot2)
library(dplyr)


# This function generates random (x, y) points within a rectangular region
# These points are used to estimate the area under a curve (Monte Carlo method)
generate_mc_points <- function(n, x_min, x_max, y_min, y_max) {
  x_coords <- runif(n, min = x_min, max = x_max)  # Random x-values
  y_coords <- runif(n, min = y_min, max = y_max)  # Random y-values
  data.frame(x = x_coords, y = y_coords)
}


# Generate datasets for different sample sizes
# We simulate increasing numbers of random points to observe convergence
n_values <- c(10, 100, 1000, 10000)
```

```r
mc_data <- lapply(n_values, function(n) {
  df <- generate_mc_points(
    n,
    x_min = -4, x_max = 4,     # x-range of the normal distribution
    y_min = 0, y_max = 0.4     # y-range covering the curve height
  )
  df$n <- n  # Store sample size for faceting
  df
}) |> bind_rows()


# Classify points relative to the curve
# Label each point as:
# - "on/below" the curve (inside area)
# - "above" the curve (outside area)
mc_data <- mc_data |>
  mutate(
    flag = ifelse(
      y <= dnorm(x, mean = 0, sd = 1),  # Compare to normal density function
      "on/below",
      "above"
    )
  )


# Create visualization
# Plot random points and overlay the true normal distribution curve
ggplot(mc_data, aes(x = x, y = y, color = flag)) +

  # Plot sampled points (Monte Carlo simulation)
  geom_point(alpha = 0.6, size = 1.5) +

  # Overlay the true normal distribution curve
  stat_function(
    fun = dnorm,
    args = list(mean = 0, sd = 1),
    xlim = c(-4, 4),
    color = "brown",
    linewidth = 1.2
  ) +

  # Assign colors to points above vs below the curve
  scale_color_manual(
    values = c("above" = "turquoise", "on/below" = "lightpink")
  ) +

  # Create small multiple panels for each sample size
```

```r
  facet_wrap(~ n, nrow = 2, labeller = label_both) +

  # Add titles and labels
  labs(
    title = "Monte Carlo Integration of the Standard Normal Distribution",
    x = "x",
    y = "Density",
    color = "Point Location"
  ) +

  # Apply clean theme
  theme_minimal() +

  # Position legend at bottom
  theme(legend.position = "bottom")
```